

Software Construction and Software Testing

TTTE6104

Software Engineering
Paradigm



Objectives

Contents

- Introduction to Software Construction
- System Design
- Programming
- System Testing

Introduction to Software Construction

- The Construction Phase in the Rational Unified Process includes:
 - System design, programming, and testing.
 - Development of different parts of the system in parallel.
 - Integration of system components during this phase.
- On completion of the Construction Phase:
 - A working software system is produced.
 - Associated documentation is prepared.
 - The system is ready for delivery to users.

System Design

- Architectural Design
 - Defines the high-level structure of the system, identifying major components and how they interact.
 - Common architectural styles: Client-server, Layered, Microservices, etc.
- Component Design
 - Decomposes the system into smaller, independent modules or components.
 - Defines the responsibilities and interfaces of each component.
- Data Design
 - Defines how data will be structured, stored, and accessed.
 - Includes designing databases, data flows, and data models.
 - Ensures data integrity, consistency, and security.
- Interface Design
 - Defines how system components will communicate with each other.
 - Ensures seamless integration between system modules and a user-friendly interface.

Architectural Design

- The first stage in the software design process.
- Architectural design is concerned with understanding how a system should be organized and designing the overall structure of that system.
- The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.
- As part of the requirements engineering process, you might propose an abstract system architecture where you associate groups of system functions or features with large-scale components or sub-systems.

Two levels of abstraction software architectures

- Architecture in the small
 - concerned with the architecture of individual programs
 - concerned with the way that an individual program is decomposed into components
- Architecture in the large
 - concerned with the architecture of complex enterprise systems that include other systems, programs, and program components

Advantages of explicitly designing and documenting software architecture

- Stakeholder communication – The architecture is a high–level presentation of the system that may be used as a focus for discussion by a range of different stakeholders.
- System analysis – Making the system architecture explicit at an early stage in the system development requires some analysis. Architectural design decisions have a profound effect on whether or not the system can meet critical requirements such as performance, reliability, and maintainability.
- Large–scale reuse – A model of a system architecture is a compact, manageable description of how a system is organized and how the components interoperate.

Architectural views

- A logical view, which shows the key abstractions in the system as objects or object classes. It should be possible to relate the system requirements to entities in this logical view.
- A process view, which shows how, at run-time, the system is composed of inter-acting processes. This view is useful for making judgments about non-functional system characteristics such as performance and availability.
- A development view, which shows how the software is decomposed for development, that is, it shows the breakdown of the software into components that are implemented by a single developer or development team. This view is useful for software managers and programmers.
- A physical view, which shows the system hardware and how software components are distributed across the processors in the system. This view is useful for systems engineers planning a system deployment.

The model– view–controller (MVC) pattern

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model. See Figure 6.3.
Example	Figure 6.4 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code and code complexity when the data model and interactions are simple.

The layered architecture pattern

Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system. See Figure 6.6.
Example	A layered model of a system for sharing copyright documents held in different libraries, as shown in Figure 6.7.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

Figure 6.6 A generic layered architecture

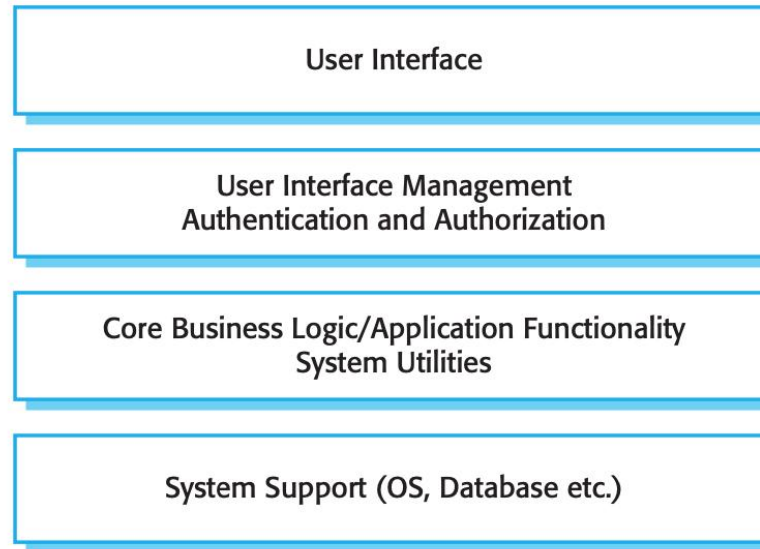
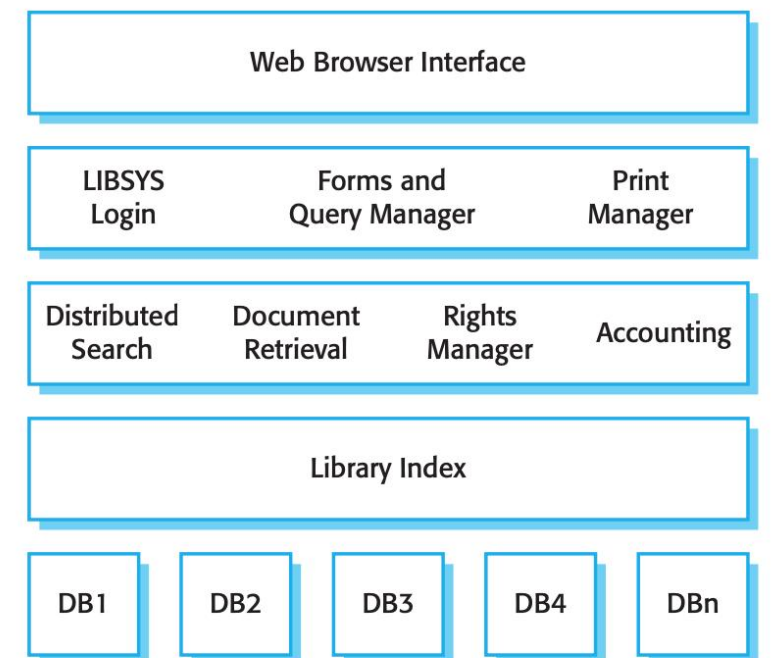


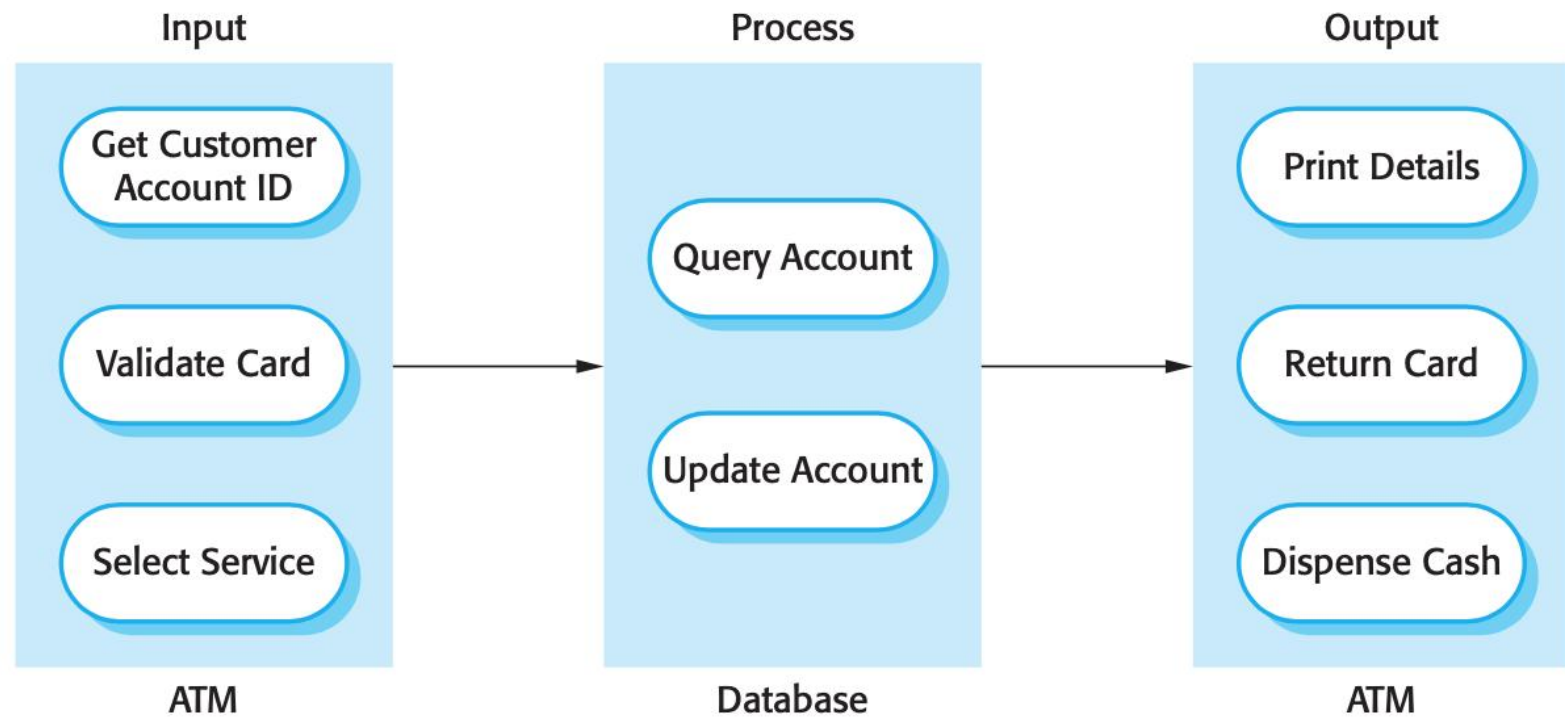
Figure 6.7 The architecture of the LIBSYS system



Other Architecture

- Repository architecture
- Client—server architecture
- Pipe and filter pattern
- Application architecture

Figure 6.15 The software architecture of an ATM system



KEY POINTS – Software Architecture

- A software architecture is a description of how a software system is organized. Properties of a system such as performance, security, and availability are influenced by the architecture used.
- Architectural design decisions include decisions on the type of application, the distribution of the system, the architectural styles to be used, and the ways in which the architecture should be documented and evaluated.
- Architectures may be documented from several different perspectives or views. Possible views include a conceptual view, a logical view, a process view, a development view, and a physical view.
- Architectural patterns are a means of reusing knowledge about generic system architectures. They describe the architecture, explain when it may be used, and discuss its advantages and disadvantages.
- Commonly used architectural patterns include Model–View–Controller, Layered Architecture, Repository, Client–server, and Pipe and Filter.
- Generic models of application systems architectures help us understand the operation of applications, compare applications of the same type, validate application system designs, and assess large–scale components for reuse.

Activity 1: Collaborative System Design Exercise

- Objective: Apply architectural, component, data, and interface design principles to a real-world problem.
- Work in the same team during Masjid Tour.
- Project scenario — Smart Operations Management System for Masjid
- Work on the following tasks:
 - Architectural Design: Choose an architectural style (e.g., client-server, microservices) and sketch a high-level design.
 - Component Design: Break down the system into major components/modules and define the responsibilities of each.
 - Data Design: Design the data structures, including database schema and data flow diagrams. Ensure they include considerations for data integrity and security.
 - Interface Design: Define how different system components will interact, ensuring smooth communication between modules.
- Presentation: Teams will present their designs, explaining the rationale behind their architectural and component choices, data structures, and interface communication.

Objectives – Software Testing

- understand the stages of testing from testing, during development to acceptance testing by system customers;
- have been introduced to techniques that help you choose test cases that are geared to discovering program defects;
- understand test–first development, where you design tests before writing code and run these tests automatically;
- know the important differences between component, system, and release testing and be aware of user testing processes and techniques.

What is testing?

- Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use.
- When you test software, you execute a program using artificial data.
- You check the results of the test run for errors, anomalies, or information about the program's non-functional attributes.

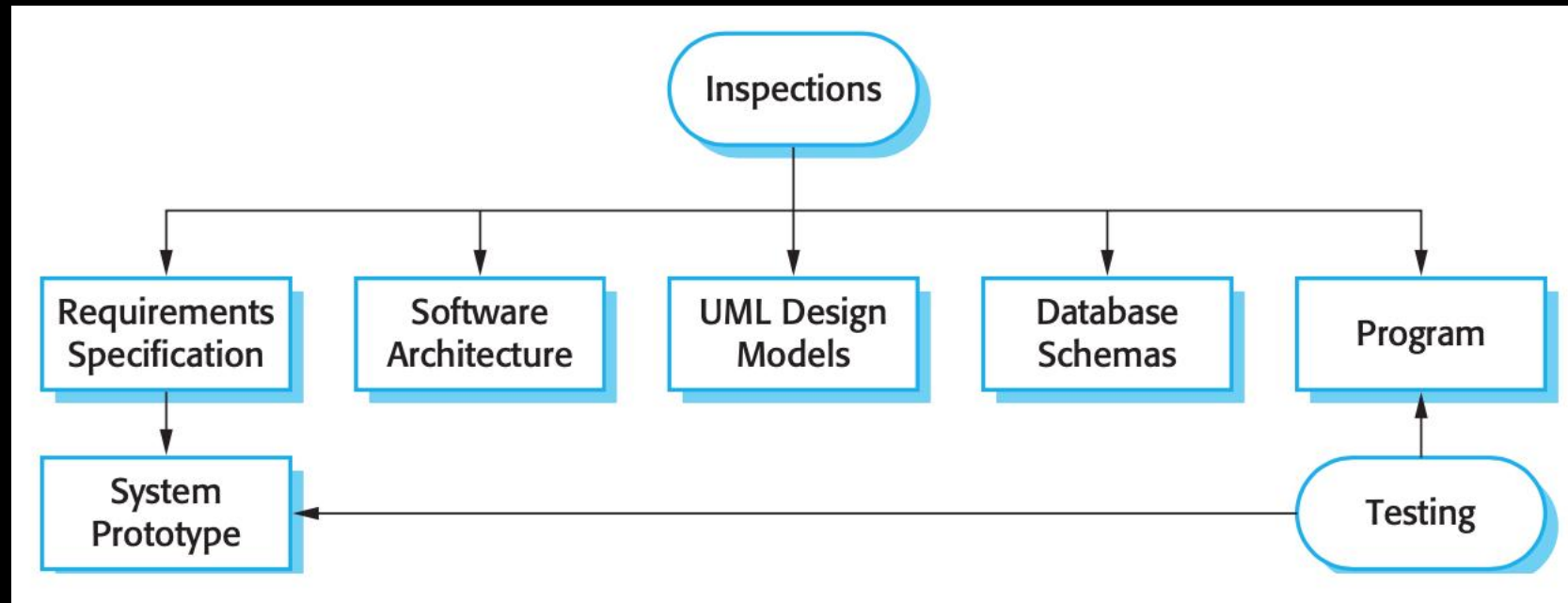
Testing process has two distinct goals:

- To demonstrate to the developer and the customer that the software meets its requirements.
 - For custom software, this means that there should be at least one test for every requirement in the requirements document.
 - For generic software products, it means that there should be tests for all of the system features, plus combinations of these features, that will be incorporated in the product release.
- To discover situations in which the behavior of the software is incorrect, undesirable, or does not conform to its specification.
 - Consequence of software defects:
 - undesirable system behavior such as system crashes, unwanted interactions with other systems, incorrect computations, and data corruption.

Verification and validation processes

- Verification and validation processes are concerned with checking that software being developed meets its specification and delivers the functionality expected by the users
- Validation: Are we building the right product?
 - To check that the software meets its stated functional and non-functional requirements
- Verification: Are we building the product right?
 - To ensure that the software meets the customer's expectations.
- Involve software inspections and reviews.
 - Inspections and reviews analyze and check the system requirements, design models, the program source code, and system tests.

Inspection & Testing



Three stages of testing

- Development testing, where the system is tested during development to discover bugs and defects. System designers and programmers are likely to be involved in the testing process.
- Release testing, where a separate testing team tests a complete version of the system before it is released to users. The aim of release testing is to check that the system meets the requirements of system stakeholders.
- User testing, where users or potential users of a system test the system in their own environment. For software products, the 'user' may be an internal marketing group who decide if the software can be marketed, released, and sold. Acceptance testing is one type of user testing where the customer formally tests a system to decide if it should be accepted from the system supplier or if further development is required.

Development testing

- Unit testing, where individual program units or object classes are tested. Unit testing should focus on testing the functionality of objects or methods.
- Component testing, where several individual units are integrated to create composite components. Component testing should focus on testing component interfaces.
- System testing, where some or all of the components in a system are integrated and the system is tested as a whole. System testing should focus on testing component interactions.

Release testing

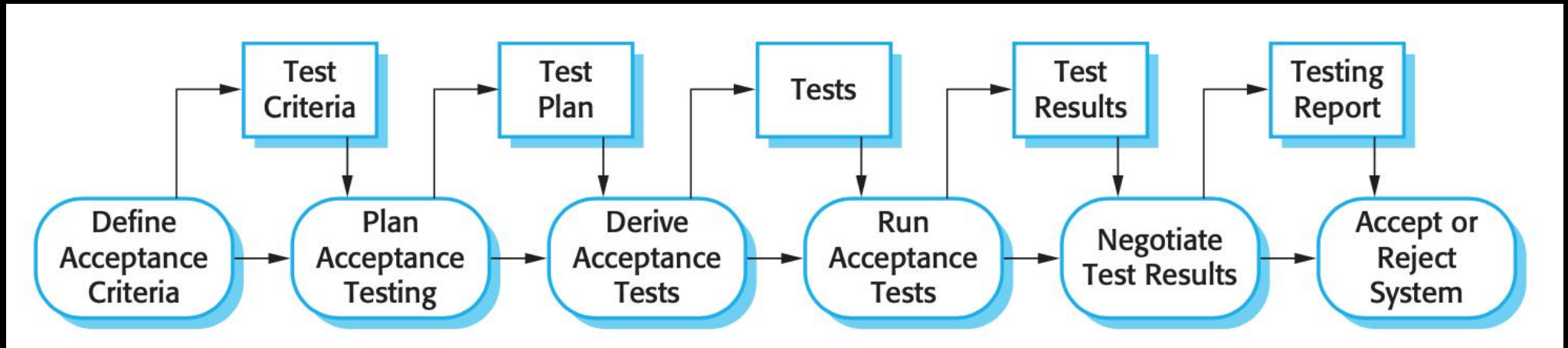
- The process of testing a particular release of a system that is intended for use outside of the development team.
- Normally, the system release is for customers and users.
- In a complex project, however, the release could be for other teams that are developing related systems.
- For software products, the release could be for product management who then prepare it for sale.

User testing

- User / customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.
- User testing is essential, even when comprehensive system and release testing have been carried out.
- The reason for this is that influences from the user's working environment have a major effect on the reliability, performance, usability, and robustness of a system.

Three different types of user testing

- Alpha testing, where users of the software work with the development team to test the software at the developer's site.
- Beta testing, where a release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
- Acceptance testing, where customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment.



The acceptance testing process

KEY POINTS — Software Testing

- Testing can only show the presence of errors in a program. It cannot demonstrate that there are no remaining faults.
- Development testing is the responsibility of the software development team. A separate team should be responsible for testing a system before it is released to customers. In the user testing process, customers or system users provide test data and check that tests are successful.
- Development testing includes unit testing, in which you test individual objects and methods; component testing, in which you test related groups of objects; and system testing, in which you test partial or complete systems.
- When testing software, you should try to ‘break’ the software by using experience and guidelines to choose types of test cases that have been effective in discovering defects in other systems.
- Wherever possible, you should write automated tests. The tests are embedded in a program that can be run every time a change is made to a system.
- Test-first development is an approach to development where tests are written before the code to be tested. Small code changes are made and the code is refactored until all tests execute successfully.
- Scenario testing is useful because it replicates the practical use of the system. It involves inventing a typical usage scenario and using this to derive test cases.
- Acceptance testing is a user testing process where the aim is to decide if the software is good enough to be deployed and used in its operational environment.

References

- Sommerville, I. (2016). Software engineering. Pearson.